

Law-Abiding Agents

Demonstrating Safe Tax Preparation with Data Flow Control

Charlie Summers*, Prajwal Raghunath*, Zhibin Shen*, Hardik Gupta*, Eric Choi*, Mayur Kulkarni*, Sally Go*, Peter Yu*, Akriti Agarwal*, Zhuo Zhang*, Oliver Kennedy†, Eugene Wu*

*Columbia University †University at Buffalo
New York, USA

{cgs2161,pr2789,zs2666,hg2770,esc2219,msk2277,yg3066,hy2846,aa5807}@columbia.edu,
zz@cs.columbia.edu,okennedy@buffalo.edu,ewu@cs.columbia.edu

ABSTRACT

Agents that act on data must not only produce correct outputs but also satisfy federal, organizational, and user-defined policies. Recent work introduced Data Flow Control (DFC), which enforces policies over how outputs are derived from inputs inside the DBMS via provenance-aware query rewriting. This demonstration presents an end-to-end system for safe data-centric automation, including policy extraction, monitoring, and violation handling. We showcase a tax agent that safely analyzes, updates, and commits data while surfacing and resolving policy violations.

PVLDB Reference Format:

Charlie Summers*, Prajwal Raghunath*, Zhibin Shen*, Hardik Gupta*, Eric Choi*, Mayur Kulkarni*, Sally Go*, Peter Yu*, Akriti Agarwal*, Zhuo Zhang*, Oliver Kennedy†, Eugene Wu*. Law-Abiding Agents. PVLDB, 14(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/reallof/data-flow-control>.

1 INTRODUCTION

Recent LLM-based agents, such as OpenAI’s Operator [6], Anthropic’s Cowork [1], and OpenClaw [7], highlight the potential for end-to-end automation over real-world tasks. Consider a *tax agent* that helps small businesses, contractors, and freelancers prepare Schedule C deductions¹. Given a set of receipts, the agent classifies expenses, computes deductions, and insert them into an expenses table that will populate tax forms. We expect such *data-centric* tasks—where agents interact primarily through database queries and updates—to become dominant in enterprise and regulated settings, where data systems serve as the system of record.

Simply completing the task in a way that appears correct to the user is not enough; the agent must also satisfy regulatory, financial, and organizational policies. Even seemingly simple tax deductions are governed by many rules. 26 US Code §274(n) requires that no more than 50% of meal expenses may be deducted. Receipt scans

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

¹19.4% of U.S. taxpayers filed Schedule C in 2022.

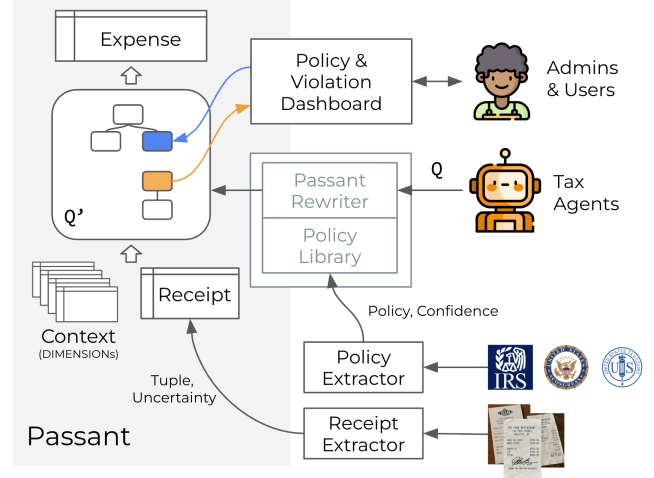


Figure 1: TAXAGENT extends Passant (in grey) with the mechanisms to support full policy life-cycle management, including asynchronous violation handling to support policy and violation monitoring dashboards; a range of context that can be referenced by policies; and supporting policy extraction confidence and data uncertainty.

are often noisy due to OCR errors, and tax rules allow limited flexibility: an aggregate policy states that deductions from low-certainty receipts may be allowed only up to a bounded total risk; beyond that, the agent should defer to a human.

Both are *data flow policies* that constrain how outputs are derived from inputs, often across multiple records and aggregations. Enforcing such policies requires reasoning about data flows during query execution, which existing approaches cannot express. Database mechanisms like integrity constraints and access controls do not constrain how data is combined and derived, while LLM-based guardrails [2, 3, 9] are external and cannot observe nor enforce tuple-level data flows. Frontier models achieve only ~40% F1 when checking simple data policies over queries and results, while incurring non-trivial latency and monetary cost *per query* [12].

To address this gap, recent work [11, 12] introduced *Data Flow Control (DFC)*, a provenance-based policy model that constrains how data is derived and propagated within a DBMS. DFC expresses policies as predicates over output tuples’ provenance annotations. The policy language captures a wide range of constraints, including

those above. For instance, the following policies enforce the meal deductions and uncertainty policies above. Each policy applies to any query that reads from Receipt and writes to Expense, and kills the query if the constraint is violated².

Example 1 (Meal Deduction Limit).

```
SOURCE Receipt AS R SINK Expense AS E
CONSTRAINT R.category != 'Meal' OR E.deduct <= 0.5
ON FAIL KILL
```

Example 2 (Aggregate Risk Bound).

```
SOURCE Receipt AS R SINK Expense AS E
CONSTRAINT SUM(E.cost * E.deduct * (1-R.qual)) <= 500
ON FAIL KILL
```

The system called Passant enforces DFC policies via query rewriting [12], which is deterministic and incurs ~0% overhead. Unfortunately, it does not address three core challenges necessary to support end-to-end agentic automation such as a tax agent.

Challenge 1: Policy lifecycle management. Even for Schedule C deductions, there are hundreds of policies; considering all of tax law, there are thousands. These cannot be manually authored so must be automatically extracted from legal text or generated by agents. As a result, policies vary in realism, and the system must incorporate confidence: high-confidence policies are strictly enforced, while low-confidence ones are monitored and validated, blurring enforcement and auditing.

Challenge 2: Contextual intervention. Law-abiding agents require policies that intervene on both data and context. Passant [12] theoretically expresses this with the **DIMENSION** clause, but it is unclear how to use it in practice. We highlight policies that reference context like agent actions, environment state, and user metadata.

Challenge 3: Monitoring and resolving violations. When a policy is violated, the system must surface the offending tuples and incorporate external decisions (e.g., from an accountant). Since DFC policies are evaluated during query execution, this requires an asynchronous mechanism to export violations and import resolutions. We model this as a pair of *asynchronous operators* that send violating tuples during execution, and receives decisions (e.g., accept, modify, or discard) that are reintegrated into the data flow.

This demonstration presents TAXAGENT, which automatically computes Schedule C deductions from receipt images and guarantees data safety with respect to tax law. We build on DFC and make the following contributions:

- An LLM-based tax policy extractor that emits confidence levels.
- Extensions to DuckDB and Passant to support asynchronous policy resolution via user-defined functions.
- We show how DFC’s policy features and our extensions are brought together in an end-to-end TAXAGENT framework that guarantees law-abiding tax handling, policy ingestion and management, and a dashboard inspecting policy violations and agent behavior.

This work shifts data systems from passive storage layers towards active enforcement substrates for safe, legal agentic automation.

²This work focuses on per-query enforcement, and we plan to address cross-query enforcement in future work.

2 BACKGROUND AND MOTIVATION

We briefly introduce Data Flow Control and then describe end-to-end policy lifecycle management for TAXAGENT.

2.1 Data Flow Control

The key idea behind DFC is that provenance polynomials [4] logically encode the derivations (data flow) of a query’s output tuples, thus controlling a query’s data flows is equivalent to constraints over these derivations.

DFC policies [12] are applied to queries that match the policy’s input relations (in **SOURCE**); for insert and update queries, the target relation is matched against relations in the **SINK** clause. For those queries, the **CONSTRAINT** is evaluated for each output tuple, which is annotated with its provenance. The **ON FAIL** clause specifies how to handle constraint violations.

Example 1 specifies U.S.C. §274(n). It matches queries that read from **Receipt** and write to **Expense**, and checks that meal receipts are deducted by at most 0.5. Any attribute in **SOURCE** and **SINK** relations may be referenced; it is evaluated over all provenance monomials (derivations) for each output tuple as an aggregate predicate. Since the output may be derived from multiple receipts, the lack of an aggregation function over **R.category** implies that the output is derived from exactly one receipt.

Example 2 penalizes the amount each expense is deducted by that receipt’s uncertainty (**qual**), and does not allow the query to execute if the total penalty exceeds a threshold (e.g., 500). Such policies that aggregate over both **SOURCE** and **SINK** are applied to all output tuples to pass or fail the entire query.

ON FAIL executes a handler when an output tuple violates the constraint. The defaults **REMOVE** a violating output from the result set (enforced as a filter condition on the rewritten query) or **KILL** the query (calls a UDF that throws an exception). Any UDF can be a handler—if the UDF returns an “edited” tuple, it replaces the violating tuple. Unfortunately, Passant only supports synchronously executed handlers; asynchronous handlers are challenging because they divert violating tuples out of the query’s data flow and converges “fixed” tuples back.

DFC also supports the **DIMENSION** clause which specifies additional relations that are one-to-one with query results and thus can be referenced in the constraint. Section 3.2 describes how this powerful mechanism can be used to flexibly inject context into policy enforcement decisions. We refer the reader to [12] for a full description of DFC features.

2.2 TAXAGENT Overview

Figure 1 provides an overview of TAXAGENT, currently implemented in DuckDB[8]. It extends Passant with mechanisms for policy and violation life-cycle management.

A key extension is to account for LLM confidences in policy and data handling. For instance, the LLM-based policy extractor described in Section 3.1 specifies whether a policy is high or low confidence. We use this to decide whether to directly enforce a policy, or first delegate violations to the dashboard. Extracted receipts also specify uncertainty that policies can reference in their **CONSTRAINT** clauses.

The tax agent issues queries that read from the database, including Receipt, and inserts deductions into the Expense relation. Passant enforces all relevant policies for a given query Q by rewriting it into Q' . We extend Passant to support asynchronous violation handlers, and introduce the pair of **send/receive** operators that communicate with external services, such as web services, users, and our dashboard (Section 3.3).

Any data that can be modeled as a relation or view can be used as context for policies to reference in their constraints. Section 3.2 describes several ways agent-relevant context can be introduced using the **DIMENSION** clause.

3 DATA FLOW CONTROL EXTENSIONS

This section describes DFC extensions necessary to handle large, diverse policies like those for TAXAGENT. They cover policy definition, expressivity, and resolution. The examples will omit the **SOURCE**, **SINK**, and **ON FAIL** clauses since they are identical; R and E will refer to Receipt and Expense respectively.

3.1 Automatic Policy Generation

In order to scalably populate the policy library, we develop an LLM-based policy extractor that takes Federal US tax law as input and outputs DFC policies and extraction confidences. We follow existing work that extracts from legal text using LLMs [5]; the key goal is not perfect extraction but to generate “sensible” policies of various confidences at scale. We plan to develop more sophisticated extraction methods in future work.

The difficulty is that legal text exceeds LLM context windows by an order of magnitude. Naive and per-section chunking do not work because policies may span chunk boundaries and cross-reference other sections. For instance 26 US Code § 274(h)(7)—“*No deduction shall be allowed under section 212 for expenses allocable to a convention, seminar, or similar meeting.*”—references investment income details in Section 212. An LLM cannot create the following desired DFC policy without including section 212 text:

```
CONSTRAINT R.type != 'Investment' OR
R.category != 'Seminar' OR E.deduct = 0%
```

Thus, we follow Sannier et al. [10] to construct a cross-reference graph. For each section, we transitively append cross-referenced text to the LLM prompt until we reach a context limit.

We use a majority vote over 3 LLM judges determine policy confidence. Each judge assesses the extracted policy and source text and returns their confidence {HIGH, LOW} that the policy reflects a rule in the text. We use Claude Opus 4.6 for policy generation and Gemini 3.1 Pro, GPT 5.4, and Claude Sonnet 4.6 for voting.

3.2 Context-awareness with **DIMENSION**

Passant [12] introduced the **DIMENSION** clause to incorporate arbitrary relations and metadata. We now use three examples to illustrate the range of agent-relevant context that can be used to enforce policies: metadata and preferences, environment and agent state, and agent trajectory.

Metadata and Preferences. Often, we wish to customize policy details based on user preferences, data residence details, database

load, or other factors. For the risk boundary policy in Example 2, users can select their own risk threshold by adding a row to a relation Thresh(uid, risk, ...). The policy then references a view ResolvedThresh that resolves to the user’s threshold when present, and otherwise falls back to a default:

```
DIMENSION ResolvedThresh AS T
CONSTRAINT SUM(E.cost * E.deduct * (1-R.qual)) <= T.risk
```

Environment and Agent State. Modern data systems increasingly support branching to isolate changes from production (e.g., main vs derived branches). At the same time, agent actions during test-time exploration mode (e.g., MCTS) should be treated differently than during commit mode. Action safety depends on the environment and agent state. As a simple example, we model this as relations EnvState(branch) and AgentState(mode):

```
DIMENSION EnvState AS E, AgentState AS A
CONSTRAINT E.branch != 'main' OR A.mode = 'commit'
```

Agent Trajectory. Just as policies may refer to data provenance, they may also refer to *action provenance*: the executed actions during an agent’s trajectory. Agent frameworks already expose this execution trace, which can be modeled as Trajectory(step, action, input, output, ...). Policies can then enforce properties over the trajectory, such as requiring the agent look up a business with BizLookup to ground decision making before writing expenses:

```
DIMENSION (SELECT COUNT(*) AS n FROM Trajectory
WHERE action = 'BizLookup') AS T
CONSTRAINT T.n >= 1
```

3.3 Asynchronous Violation Handlers

It is important that administrators and users can use dashboards to monitor the status of policies, their enforcement, and violations in real time—for observability, so users can verify or fix violations interactively, and to manage low-confidence policies. For instance, low-confidence policies may not correctly extracted. Rather than remove the policies outright, we evaluate them as normal, but send their violations to a dashboard for user review to decide whether the policy or violation is correct.

Passant does not support this functionality because it handles violations synchronously, so the entire query execution would block on each violation sent to the dashboard until it returns. Instead, we need to handle violations asynchronously so the query can fully execute, all violations be identified, and then the query results can be streamed or blocked until violations are resolved, based on the desired policy.

We developed a DuckDB extension that introduces the **AsyncSend (S)** and **AsyncReceive (R)** operators to support external services as violation handlers (Figure 2). The handler is defined with a connection string that Passant uses to establish and maintain a web-socket connection. Passant rewrites the input query Q to logically split the data flow based on whether the output tuple satisfies the policy constraint P ; those that fail are sent to **AsyncSend**, which

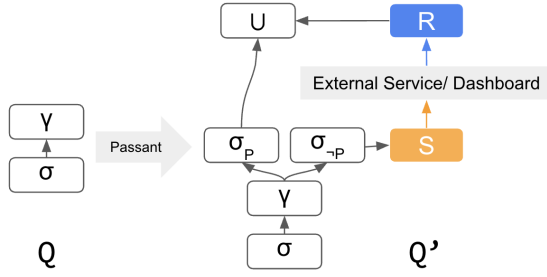


Figure 2: **Passant** is extended to handle violations asynchronously with the **AsyncSend** and **AsyncReceive** operators.

forwards the violation along with the policy and query ID to the connection. Responses are received by **AsyncReceive**; nulls are dropped while tuples are emitted to its parent operator. A UNION operator combines this data flow with the non-violating output tuples to form the result set.

During execution, the rewritten query returns non-violating results then streams cleaned violations as they are received from the dashboard. This allows the agent to pipeline its execution with violation handling. We implement **AsyncSend** and **AsyncReceive** as custom operators instead of extending DuckDB’s http operators because the existing operators send synchronous http requests.

4 DEMONSTRATION

Participants will interact with the four workflows described below and shown in Figure 3.

Policy Extraction. Participants begin by extracting policies from natural language. They copy and paste legal text into the chat interface, and the policy extractor extracts cross-references, constructs extraction prompts, and produces policies and confidence scores. Participants can edit the policies, confidences, or write custom policies. The first view presents the policy library, including each policy’s definition, confidence, coverage (number of queries or tuples evaluated), and violation statistics.

Receipt Submission. Participants then submit receipts in the chat—either a provided set or their own receipts. As the agent processes receipts to determine tax deductions via queries, they are validated against policies.

Violation Management. As the agent runs, participants monitor a live dashboard. The right-side view lists policies and violations. It shows distributions over queries and affected tuples. Participants can inspect an individual violation to see the tuple, policy, and the originating legal text. They can then reject the violation, delete the tuple, or edit the data to bring it into compliance.

Participants will see several violations including the following. First, a meal receipt is incorrectly deducted at 100%, violating the 50% limit (Example 1). Second, the agent applies a 100% deduction to a seminar for a low-confidence policy (26 U.S.C. § 274(h)(7)), and the user will review and fix the violation while the agent continues. Finally, after all receipts are processed, the aggregate risk policy (Example 2) will fail due to excessive uncertainty, and participants will identify and correct the problematic receipts.

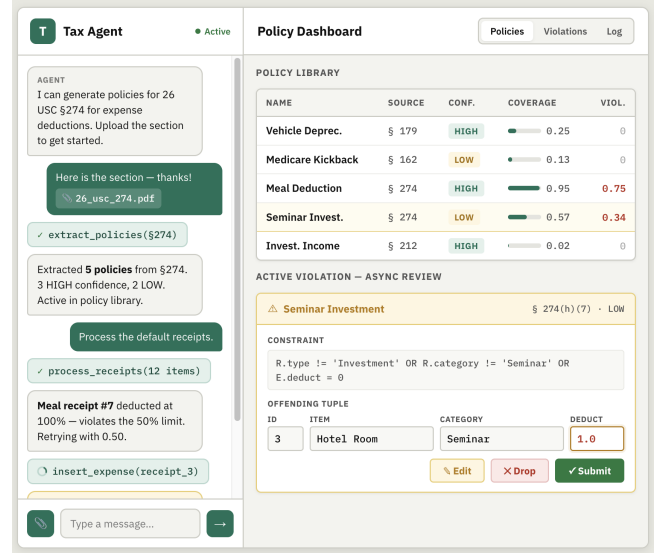


Figure 3: The **TAXAGENT** chat interface (left) is used to upload receipts, generate policies from natural language, and watch the agent as it deducts expenses. The policy dashboard (right) helps inspect and drill into policies and resolve violations.

Agent Creation. Finally, participants can vibe code a tax agent, and are encouraged to explore edge cases and non-compliant behaviors. They can see how the system detects and surfaces policy violations; they can manually submit unsafe queries too.

REFERENCES

- [1] Anthropic. 2026. The Future of AI at Work: Introducing Cowork. <https://www.anthropic.com/webinars/future-of-ai-at-work-introducing-cowork>.
- [2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073* (2022).
- [3] Dynamo AI. 2024. Dynamo AI: Guardrails for Enterprise AI Systems. <https://www.dynamo.ai/>.
- [4] Todd J Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance semirings. In *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 31–40.
- [5] Gabriele Lorenzo, Aldo Pietromatera, and Nils Holzenberger. 2025. Translating Tax Law to Code with LLMs: A Benchmark and Evaluation Framework. In *Proceedings of the Natural Legal Language Processing Workshop 2025*. 31–47.
- [6] OpenAI. 2025. Introducing Operator. <https://openai.com/index/introducing-operator/>.
- [7] OpenClaw. 2026. Introducing OpenClaw. <https://openclaw.ai/blog/introducing-openclaw>.
- [8] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. <https://doi.org/10.1145/3299869.3320212>.
- [9] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. In *EMNLP Demo*.
- [10] Nicolas Sannier, Morayo Adedjouma, Mehrdad Sabetzadeh, and Lionel Briand. 2017. An automated framework for detection and resolution of cross references in legal texts. *Requirements Engineering* 22, 2 (2017), 215–237.
- [11] Charlie Summers, Haneen Mohammed, and Eugene Wu. 2025. Please Don’t Kill My Vibe: Empowering Agents with Data Flow Control. *arXiv preprint arXiv:2512.05374* (2025).
- [12] Charlie Summers and Eugene Wu. 2026. Data Flow Control: Data Safety Policies for AI Agents. *ArXiv*.